Software Requirements Specification

A Mobile Application for Facility Management

Team
Team 7

Members
Habiba Farouk 13007597
Jamila Riad 13007552
Andrew Nabil 13007493
Jana Elshoura 10000520
Ahmed Sherif 13001195
Pierre Ashraf 13005443
Rana Magdy 13004373

Tutorial Details
Tutorial No. 2
Moamen Alaa

1. Introduction

This Software Requirements Specification (SRS) describes the requirements of the GIU Campus Maintenance Mobile Application.

The purpose of this document is to define the system's objectives, intended users, and overall scope prior to development. It establishes a clear and shared understanding between the development team, stakeholders, and academic evaluators regarding what the system is expected to achieve.

The system provides a centralized digital platform that enables members of the GIU community to report infrastructure, maintenance, sustainability, and cleanliness issues on campus. It also allows the Facility Management team to manage, assign, monitor, and resolve reported issues in a structured and efficient manner.

The intended audience of this document includes the development team, Facility Management staff, project stakeholders, testers, and academic evaluators.

1.1 Product Vision & Scope

Product Vision

The vision of the GIU Campus Maintenance Mobile Application is to create a user-friendly and structured mobile platform that improves communication between community members and the Facility Management team.

The system aims to digitize the maintenance reporting workflow, reduce response time, and enhance transparency in handling campus issues.

Product Scope

The system will provide the following core functionalities:

- Allow community members to submit maintenance tickets including photo, description, location details, and issue category.
- Enable Facility Management staff to review submitted tickets, assign them to maintenance workers, and update ticket status.
- Allow maintenance workers to receive assigned tasks and mark them as completed.
- Provide visibility of ticket status updates to ensure transparency for users.

The solution will be developed using React Native (Expo) for the mobile application, Node.js with RESTful APIs for the backend, PostgreSQL for data storage, and cloud-based services for image handling.

1.2 Definitions and Acronyms

SRS: Software Requirements Specification.
Ticket: A digital record representing a maintenance issue.
RBAC: Role-Based Access Control.
FM Team: Facility Management Team.
User: Any authenticated person using the system.
Community Member: Student, staff, or faculty submitting issues.
Worker: Maintenance personnel assigned to resolve tickets.
API: Application Programming Interface used for communication between the mobile application and backend services.
Push Notification: Real-time alerts sent to users' mobile devices about ticket updates.
Authentication: The process of verifying the identity of a user before granting access.
Authorization: The process of granting permissions based on user roles.
Status: The current state of a ticket (e.g., Pending, Assigned, In Progress, Resolved).
Maintenance Workflow: The process from ticket creation until issue resolution.
Geolocation: The process of identifying the physical location of a device using GPS, Wi-Fi, or mobile network data.
UML (Unified Modeling Language): A standardized visual modeling language used to represent the structure, behavior, and interactions of a software system through diagrams.

2. Functional Requirements

This section describes what the system should do through user stories and detailed functional requirements per user role.

2.1 User stories

1.Community Member:

US-CM-01:
As a Community Member, I should be able to create an account so that I can log in to the application.

US-CM-02:
As a Community Member, I should be able to log in using my email and password so that I can access the application.

US-CM-03:
As a Community Member, I should be able to submit an issue with its details (location, photo, and category) so that the problem can be reported accurately.

US-CM-04:
As a Community Member, I should be able to add a description to the issue so that the worker understands the problem clearly.

US-CM-05:
As a Community Member, I should be able to view my submitted issues so that I can track their status.

US-CM-06:
As a community member, I want to edit or cancel a ticket before it is assigned, so that I can correct mistakes.

US-CM-07:
As a Community Member, I should be able to log out so that my data remains secure on shared devices.

US-CM-08:
As a Community Member, I should be able to view the status of my issue (Pending, In Progress, Resolved) so that I know the progress of my request.

2.Facility Management:

US-FM-01:
As a Facility Manager, I should be able to create an account so that I can log in to the application.

US-FM-02:
As a Facility Manager, I should be able to log in using my email and password so that I can access the application.

US-FM-03:
As a Facility Manager, I should be able to log out so that my data remains secure on shared devices.

US-FM-04:
As a Facility Manager, I should be able to view all submitted issues so that I can assign them to workers.

US-FM-05:
As a Facility Manager, I should be able to filter and search tickets based on status, location, issue category, submission date, or assigned worker.

US-FM-06:
As a Facility Manager, I should be able to review newly submitted tickets (status: Issued) in a prioritized dashboard view.

US-FM-07:
As a Facility Manager, I should be able to update an issue’s status so that the system reflects the current state of the issue.

US-FM-08:
As a Facility Manager, I should be able to assign an issue to a worker so that the worker can start fixing it.

US-FM-09:
As a Facility Management supervisor, I should be able to reassign tickets when necessary, so that delayed or unresolved issues are handled properly.

US-FM-10:

As a Facility Manager, I should be able to close an issue after it is resolved so that the system reflects that the issue is completed.

3.Worker:

US-W-01:
As a Worker, I should be able to create an account so that I can log in to the application.

US-W-02:
As a Worker, I should be able to log in using my email and password so that I can access the application.

US-W-03:
As a Worker, I should be able to log out so that my data remains secure on shared devices.

US-W-04:
As a Worker, I should be able to view my assigned issues so that I can start working on them with full issue details (description, location, image, and category) so that I clearly understand the task.

US-W-05:

As a Worker, I should be able to comment on an issue to indicate that the work is finished so that the facility manager can update its status to completed.

US-W-06:

As a Worker, I should be able to mark an issue as “In Progress” so that the Facility Manager and community member know that it is being worked on.

US-W-07:

As a Worker, I should be able to update the ticket status from InProgress → Finished once the maintenance task is completed.

US-W-08:
As a Worker, I should be able to upload a photo after completing the issue so that the Facility Manager can verify the work quality.

4.System Admin (optional for Now):

US-SA-01:

As a System Admin, I should be able to log in so that I can access the application.

US-SA-02:

As a System Admin, I should be able to log out so that my data remains secure on shared devices.

US-SA-03:

As a System Admin, I should be able to activate or deactivate any account so that I can ensure all users follow the system rules.

US-SA-04

As a System Admin, I should be able to view all registered users so that I can manage the system users.

2.2 Functional Requirements by User Role

User Role 1: Community Member

FR-CM-01: Account Creation
The system shall allow a Community Member to create an account by providing required information (e.g., name, email, and password).

FR-CM-02: User Authentication
The system shall allow a Community Member to log in using a valid email and password.

FR-CM-03: Issue Submission
The system shall allow a Community Member to submit an issue including the following details:

- Location
- Photo
- Category
- Description

FR-CM-04: Issue Description
The system shall allow a Community Member to add a textual description when submitting an issue.

FR-CM-05: View Submitted Issues
The system shall allow a Community Member to view a list of issues they have submitted.

FR-CM-06: Edit or Cancel Issue

The system shall allow users to edit or cancel a submitted ticket only while its status remains **Issued** (before it has been assigned).

FR-CM-07: Logout
The system shall allow a Community Member to log out of the application.

FR-CM-08: View Issue Status
The system shall display the current status of an issue to the Community Member.
Possible statuses include:

- Pending
- In Progress
- Resolved

User Role 2: Facility Management (FM) Team

FR-FM-01: Account Creation
The system shall allow a Facility Manager to create an account.

FR-FM-02: User Authentication
The system shall allow a Facility Manager to log in using a valid email and password.

FR-FM-03: Logout
The system shall allow a Facility Manager to log out of the application.

FR-FM-04: View All Issues
The system shall allow a Facility Manager to view all submitted issues in the system.

FR-FM-05: Filter and Search Tickets
The system shall allow FM staff to filter and search tickets based on status, location, issue category, submission date, or assigned worker.

FR -FM-06: Review New Tickets
The system shall allow FM staff to review newly submitted tickets (status: Issued) in a prioritized dashboard view.

FR-FM-07: Update Issue Status
The system shall allow a Facility Manager to update the status of an issue.

FR-FM-08: Assign Issue to Worker
The system shall allow a Facility Manager to assign an issue to a specific worker.

FR-FM-09: Reassign Tickets
The system shall allow a Facility Manager to reassign an issue to a new worker if there is delay or unresolved issues.

FR-FM-10: Close Issue
The system shall allow a Facility Manager to close an issue once it has been resolved.

User Role 3: Worker

FR-FM-01: Account Creation
The system shall allow a Facility Manager to create an account.

FR-W-02: User Authentication
The system shall allow Workers to log in using their email and password.

FR-W-03: Logout
The system shall allow Workers to log out of the application to maintain data security on shared devices.

FR-W-04: View Assigned Issues
The system shall allow Workers to view only tickets assigned to them, including full details such as description, location, images, and issue category.

FR-W-05: Comment on Issue
The system shall allow a Worker to add comments to an issue.

FR-W-06: Update Task Status to InProgress
The system shall allow Workers to mark an assigned ticket as InProgress when they start working on it.

FR-W-07: Mark Task as Finished
The system shall allow Workers to update the ticket status from InProgress → Finished once the maintenance task is completed.

FR-W-08: Upload Completion Photo
The system shall allow Workers to upload a photo after completing a ticket, so the Facility Management team can verify the work.

User Role 4:System Admin Functional Requirements (Optional)

FR-SA-01: Admin Authentication
The system shall allow a System Admin to log in.

FR-SA-02: Logout
The system shall allow a System Admin to log out.

FR-SA-03: Account Management
The system shall allow a System Admin to activate or deactivate user accounts.

FR-SA-04: View Registered Users
The system shall allow a System Admin to view all registered users in the system.

2.3 Ticket Status Workflow

The system shall implement a structured ticket lifecycle to ensure proper tracking, accountability, and controlled progression of maintenance tasks.

Each ticket shall follow a predefined status sequence and shall only move forward according to authorized role permissions. The defined workflow is as follows:

Issued → Assigned → InProgress → Finished → Finalized

Issued

The ticket has been submitted by a Community Member and is awaiting review by the Facility Management (FM) Team. At this stage, the user may edit or cancel the ticket.

Assigned

The FM Team has reviewed the ticket and assigned it to a maintenance worker. The worker receives a notification and becomes responsible for handling the issue.

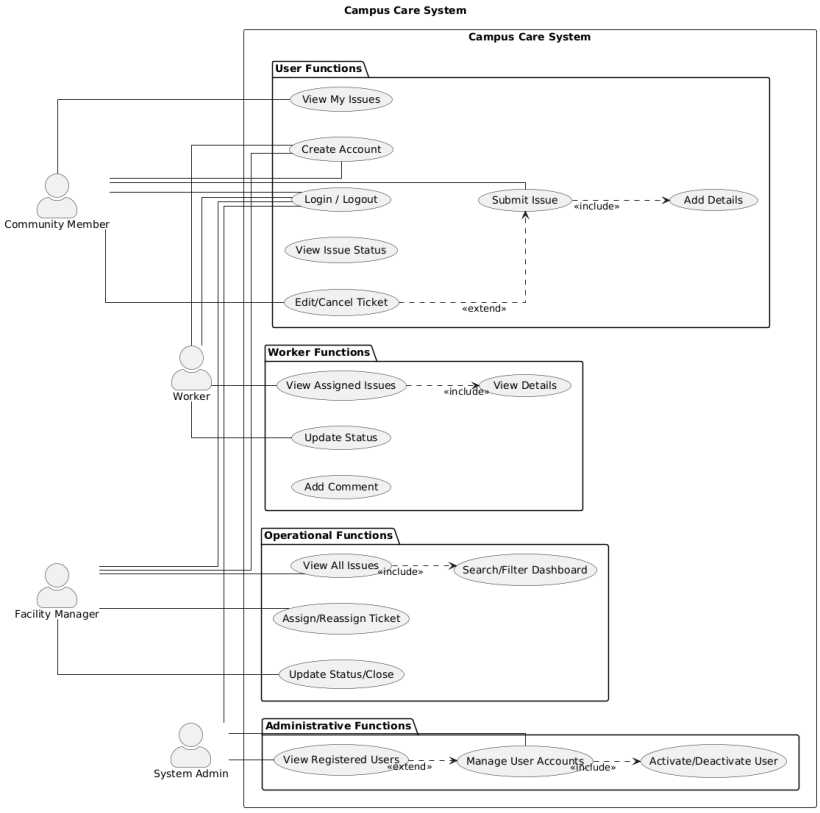
InProgress

The assigned worker has started working on the issue and has updated the ticket status accordingly.

Finished

The worker has completed the required maintenance task and may provide resolution notes. The ticket awaits verification by the FM Team.

2.4 UML Use-case Diagram



3. Non-Functional Requirements

This section defines the quality attributes and operational constraints of the GIU Campus Maintenance Mobile Application.

• **Response Time:** Standard system operations (login, ticket submission, status viewing) shall have page load times under three seconds under normal load conditions.

• **Concurrent Users:** The system shall support at least 500 simultaneous users without significant performance degradation.

• **API Security:** The backend shall implement secure authentication, HTTPS encryption, rate limiting, role-based access control (RBAC), and strict input validation to protect against unauthorized access and common vulnerabilities.

• **Backup & Recovery:** The system shall perform automated daily database backups and maintain a disaster recovery mechanism to prevent data loss.

• **Usability:** The application shall provide an intuitive and user-friendly interface such that maintenance tickets (including photo and location) can be submitted in under one minute and within five interaction steps.

• **Reliability:** The system shall maintain 99% uptime and handle image uploads robustly, including under unstable network conditions, ensuring no data corruption or loss.

• **Compatibility:** The mobile application shall operate on both Android and iOS platforms and follow RESTful standards for backend communication.

4. Constraints

This section describes the limitations and restrictions that may affect the design, development, and implementation of the GIU Campus Maintenance Mobile Application.

4.1 Technical Constraints

The system must be developed using the technologies defined in the project scope. The mobile application will be implemented using **React Native (Expo)**, while the backend will be built using **Node.js** with RESTful APIs. Data will be stored using **PostgreSQL**.
Since the project follows a predefined technology stack, alternative frameworks or database systems will not be considered during development.
Image uploads must rely on cloud-based storage solutions, which may introduce limitations related to file size, storage capacity, or network dependency.

4.2 Platform Constraints

The application is intended to run on Android and iOS mobile devices. As a result, the design must be compatible with different screen sizes and device capabilities.
The system requires an active internet connection for submitting tickets, updating statuses, and receiving notifications. Offline functionality is not supported in the current version.
Push notifications depend on third-party mobile operating system services, which may impose limitations outside the control of the development team.

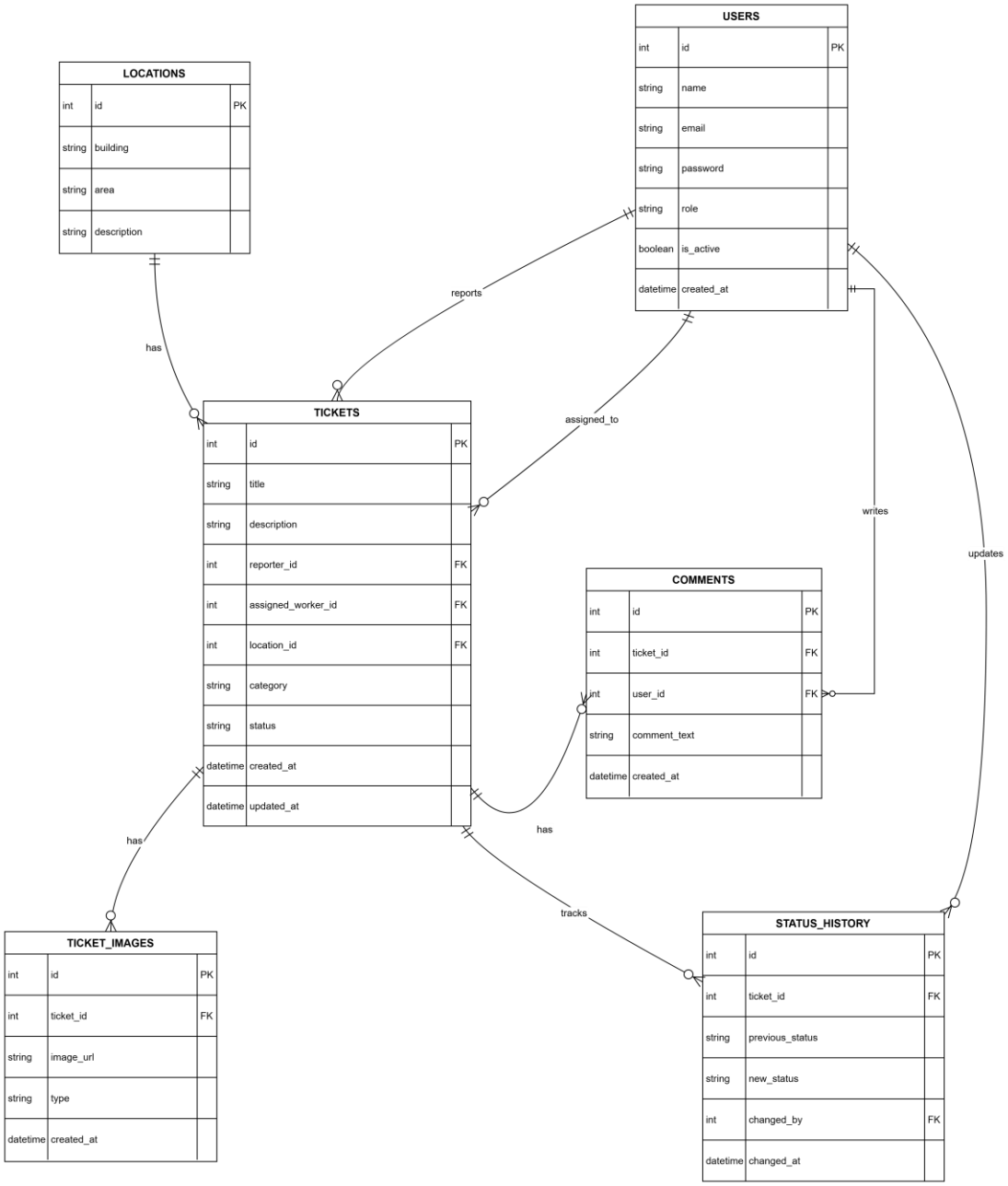
4.3 Security Constraints

The system must implement Role-Based Access Control (RBAC) to ensure that users can only perform actions permitted by their assigned role.
Authentication is required before accessing any core functionality of the system. Additionally, ticket status transitions must strictly follow the defined workflow and cannot be bypassed by unauthorized users.
Security mechanisms must comply with standard API protection practices such as input validation and access control.

4.4 Project Constraints

The system is being developed within an academic semester timeframe. Therefore, development time and resources are limited.
The project is intended for academic evaluation and demonstration purposes. It is not designed for full-scale production deployment or long-term institutional integration at this stage.

5. Entity Relation Diagram



6. Technology Stack

This section describes the technologies and technical components used to design and implement the GIU Campus Maintenance Mobile Application.

6.1 Frontend

The mobile application will be developed using **React Native (Expo)**. This framework allows cross-platform development, ensuring that the application runs smoothly on both Android and iOS devices using a single codebase.

6.2 Backend

The backend will be implemented using **Node.js**, following a RESTful API architecture. The backend will handle business logic, authentication, ticket processing, and communication between the mobile application and the database.
RESTful APIs will be used to ensure structured and secure data exchange between the frontend and backend components.

6.3 Database

The system will use PostgreSQL as a relational database management system. PostgreSQL is suitable for handling structured data such as user accounts, tickets, roles, and status records while maintaining data integrity through relationships and constraints.

Additionally, Supabase will be used as the platform to manage the database, provide authentication services, and facilitate real-time data interactions, simplifying backend development and integration.

6.4 Cloud Services

Cloud-based storage services will be used to manage uploaded ticket images. This ensures scalability and prevents overloading the main application server with large media files.
The system will also rely on cloud infrastructure for secure hosting and backup support.

6.5 Performance and Reliability Requirements

To ensure a smooth user experience, the system must meet the following technical standards:

- **Response Time:** Page load times should not exceed three seconds under normal operating conditions.
- **Concurrent Users:** The system should support at least 500 simultaneous users without performance degradation.
- **API Security:** The backend must implement rate limiting, authentication mechanisms, and input validation to protect against misuse and unauthorized access.
- **Backup and Recovery:** Automated daily database backups must be performed, and a basic disaster recovery plan should be in place to prevent data loss.

7. Future Scope of the Project

Although the current version of the GIU Campus Maintenance Mobile Application fulfills the core requirements of ticket submission and management, several enhancements may be considered in future versions to further improve functionality, efficiency, and user experience.

One potential enhancement is the introduction of a **priority classification system**, allowing tickets to be marked as low, medium, or high priority. This would help the Facility Management team address urgent issues more effectively.

Another potential enhancement is the integration of a real-time notification system to keep users informed about ticket status updates, approvals, or resolutions. This could be implemented using push notifications through platforms like Supabase, ensuring timely communication, and improved user engagement.

Another possible improvement is the integration of **advanced analytics and reporting dashboards**. These dashboards could provide insights into response times, frequently reported issues, and overall system performance, helping management make data-driven decisions.

Future versions may also include **AI-based image recognition**, where uploaded photos are automatically analyzed to suggest issue categories. This would simplify ticket submission and reduce categorization errors.
The addition of a **real-time chat feature** between users and Facility Management staff could enhance communication and allow faster clarification of reported issues.

A **web-based administrative portal** could also be developed to complement the mobile application, providing more advanced management tools for supervisors.

Other possible extensions include:

- Multi-language support for international users
- Integration with campus maps for more precise geolocation
- Service Level Agreement (SLA) tracking to monitor resolution time performance
- Offline ticket drafting with automatic synchronization once internet access is restored
  These enhancements are not part of the current implementation but may be considered in future development phases to expand the system’s capabilities and long-term impact.